

OSFF Milestone 1 — Verification Overview

1. What is in the repository

A reproducible fuzzing and verification harness for embedded firmware parsers.

Includes:

- Deterministic malformed-input corpus (39 cases)
- Two parser variants: safe and intentionally vulnerable
- Fuzz harness execution engine
- Machine-checkable evidence artifact (JSON)
- Validator enforcing artifact correctness
- One-command reproducibility (Docker-based)

2. What Milestone 1 (M1) demonstrates

M1 demonstrates a reproducible workflow for verifying parser recovery under malformed input.

Specifically:

- The safe parser returns to a functional IDLE state after all malformed inputs
- The vulnerable parser exhibits detectable behavioral divergence
- Results are captured in a structured evidence artifact
- Validation is machine-checkable and deterministic

3. What “PASS” means

PASS indicates that the verification workflow executed correctly and produced a valid artifact.

Important:

- PASS does not mean both parsers are correct

- Defect signals are visible within the artifact

(e.g., divergence on zero_length_valid_chk)

4. How this satisfies Milestone 1

M1 requirements are met:

- Reproducible execution ✓
- Defined wedge/failure conditions ✓
- Deterministic test corpus ✓
- Evidence artifact ✓
- Validator ✓
- Independent rerun capability ✓

5. How to verify (one command)

```
git clone https://github.com/repozilla2/wedgebench-osff-m1.git cd wedgebench-osff-m1 git checkout osff-m1.3 docker compose run sentinel-m1
```

Expected output:

- Parser 'safe': ✓ PASS
- Parser 'vuln': ✓ PASS
- Result: PASS

6. What this does NOT claim

- Not a full safety proof
- Not exhaustive fuzz coverage
- Not device-level certification

This provides:

Bounded, reproducible evidence of parser recovery behavior

7. One-line summary

M1 delivers a reproducible, machine-checkable verification workflow that allows independent reviewers to confirm parser recovery behavior under malformed input.